A Brief Description of UART

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

UART Protocol — Serial Communication, 8N1 Frame Format, and Baud Rate

2. Objective

By the end of this module, learners will be able to:

- Explain the UART (Universal Asynchronous Receiver Transmitter) protocol
- Describe the 8N1 frame format (1 start bit, 8 data bits, 1 stop bit)
- Calculate baud rate and clock cycles per bit at any frequency
- Trace UART TX and RX FSM states in Verilog RTL
- Run standalone UART simulation and verify loopback functionality

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

UART is the simplest way for two chips to talk to each other using just **two wires**: - **TX** (Transmit): Send data - **RX** (Receive): Receive data

Unlike most digital signals which are synchronized with a clock, UART uses **agreed-upon timing** (the baud rate) — like two people speaking at an agreed pace without a metronome.

A "byte" over UART looks like this on the wire:

```
Idle  Start  D0  D1  D2  D3  D4  D5  D6  D7  Stop  Idle
 1      0    ?   ?   ?   ?   ?   ?   ?   ?    1     1
```

- **Idle:** Wire sits at 1 (HIGH)
- **Start bit:** Always 0 — alerts the receiver that a byte is coming
- **8 data bits:** Sent LSB first (bit 0 first, bit 7 last)
- **Stop bit:** Always 1 — signals end of frame

3.2 Intermediate — Engineering View

Baud Rate = number of bits per second. Common values: 9600, 115200 baud.

At 115200 baud with a 50 MHz clock:

```
CLKS_PER_BIT = 50,000,000 / 115,200 = 434 clock cycles per bit
```

This constant `434` appears directly in the RTL:

```
// From rtl/uart_tx.v
localparam integer CLKS_PER_BIT = CLK_FREQ / BAUD_RATE;
// 50_000_000 / 115_200 = 434
```

UART TX FSM states:

```
IDLE → START → DATA (bits 0–7) → STOP → IDLE
```

UART RX challenge: The receiver has no shared clock with the sender. It must: 1. Detect the falling edge of the start bit 2. Wait **half a bit period** (217 cycles) to sample at the centre of the start bit 3. Then sample every **434 cycles** to hit the centre of each data bit

This mid-bit sampling is critical — timing errors accumulate differently if you sample at the edges.

3.3 Advanced — Signal/Architecture Level

TX Module (`rtl/uart_tx.v`) — Full FSM

```
State    Action                          Condition to advance
IDLE     tx=1 (idle high)                valid=1 → latch data → goto START
START    tx=0 (start bit)                clk_cnt == 433 → goto DATA
DATA     tx=shift_reg[0], shift right clk_cnt == 433 and bit_cnt==7 → goto
STOP
STOP     tx=1 (stop bit)                 clk_cnt == 433 → goto IDLE
```

Signal definitions:

```
input  [7:0] data;   // Byte to transmit
input        valid;  // Pulse: data is ready to send
output       ready;  // High when FSM is IDLE (tx is ready)
output reg   tx;     // Serial output
```

RX Module (`rtl/uart_rx.v`) — Full FSM

```
State    Action                                  Condition to advance
IDLE     rx_s=0 detected → start                 falling edge detected → goto
START
START    wait CLKS_HALF_BIT (217 cycles)         if rx_s still 0 → goto DATA
(glitch filter)
DATA     sample rx_s each CLKS_PER_BIT           bit_cnt==7 → goto STOP
STOP     wait CLKS_PER_BIT                       if rx_s=1 → data_out=shift_reg,
data_valid=1
```

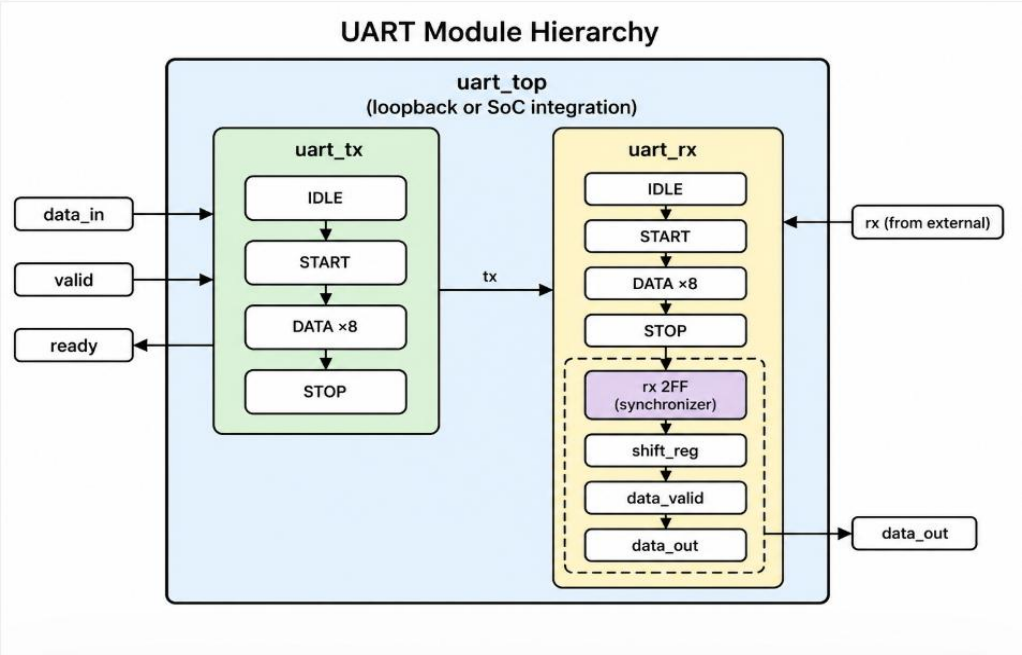
2-FF Synchronizer (metastability protection):

```
always @(posedge clk or posedge reset) begin
    rx_sync0 <= rx;       // First flip-flop
    rx_sync1 <= rx_sync0; // Second flip-flop
end
wire rx_s = rx_sync1;     // Synchronized RX input
```

This prevents metastability when the asynchronous `rx` input crosses clock domains.

4. Architecture / Concept

UART Module Hierarchy



Loopback Test Configuration

In standalone UART testing:

```
uart_top.uart_tx_out ──→ uart_top.uart_rx_in   (internally looped)
```

The testbench `tb/tb_uart_top.v` sends a byte through TX, which loops back to RX, and verifies the received byte matches.

Baud Rate Timing Diagram (115200 baud @ 50 MHz)

```
Byte: 'A' = 0x41 = 0b00100001

Wire: 1 0 1 0 0 0 0 1 0 0 1  1
      ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑  ↑
      I S D0D1D2D3D4D5D6D7 T  I
      d t                 o  d
      l a                 p  l
      e r                    e
```

Each bit lasts 434 clock cycles (8.68 µs at 50 MHz).

5. Implementation

Files in This Module

File	Role
`rtl/uart_tx.v`	UART Transmitter — 8N1, FSM-based, parameterized baud rate
`rtl/uart_rx.v`	UART Receiver — 8N1, mid-bit sampling, 2-FF synchronizer
`rtl/uart_top.v`	Top wrapper for standalone UART test (TX + RX connected)
`tb/tb_uart_top.v`	Testbench driving bytes through UART TX and verifying RX

UART Parameters (from RTL)

```
parameter CLK_FREQ  = 50_000_000;   // 50 MHz system clock
parameter BAUD_RATE = 115_200;      // 115200 baud
// Derived: CLKS_PER_BIT = 50_000_000 / 115_200 = 434
```

Key Design Decisions

1. **ASIC-clean:** All registers reset via posedge reset — no `= value` inline initializers
2. **2-FF synchronizer:** `rx_sync0 → rx_sync1` prevents metastability on async RX input
3. **Mid-bit sampling:** RX waits `CLKS_HALF_BIT - 1` before sampling start bit, then samples every `CLKS_PER_BIT` — robust against ±2.5% timing variation
4. **Glitch filter:** If RX goes high during START state (before half-bit timeout), FSM returns to IDLE (false start rejection)

6. Lab / Hands-on Section

Step 1: Run UART Standalone Simulation

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc
make uart
```

This command: 1. Cleans `obj_uart/` 2. Compiles: `uart_tx.v + uart_rx.v + uart_top.v + tb_uart_top.v` 3. Runs `./obj_uart/sim_uart` → produces `tb_uart.vcd` 4. Opens GTKWave

Expected Terminal Output

```
------------------------------------------------
 UART Standalone Simulation
------------------------------------------------
[UART TB] Sending byte: 0x48 ('H')
[UART TB] Received: 0x48 ('H') ← PASS
[UART TB] Sending byte: 0x65 ('e')
[UART TB] Received: 0x65 ('e') ← PASS
```

```
...
[UART TB] String "hello deepak" verified: ALL PASS
```

Step 2: View UART Waveform

```
gtkwave tb_uart.vcd &
# or use pre-saved layout:
gtkwave tb_uart.vcd -a uart.gtkw &
```

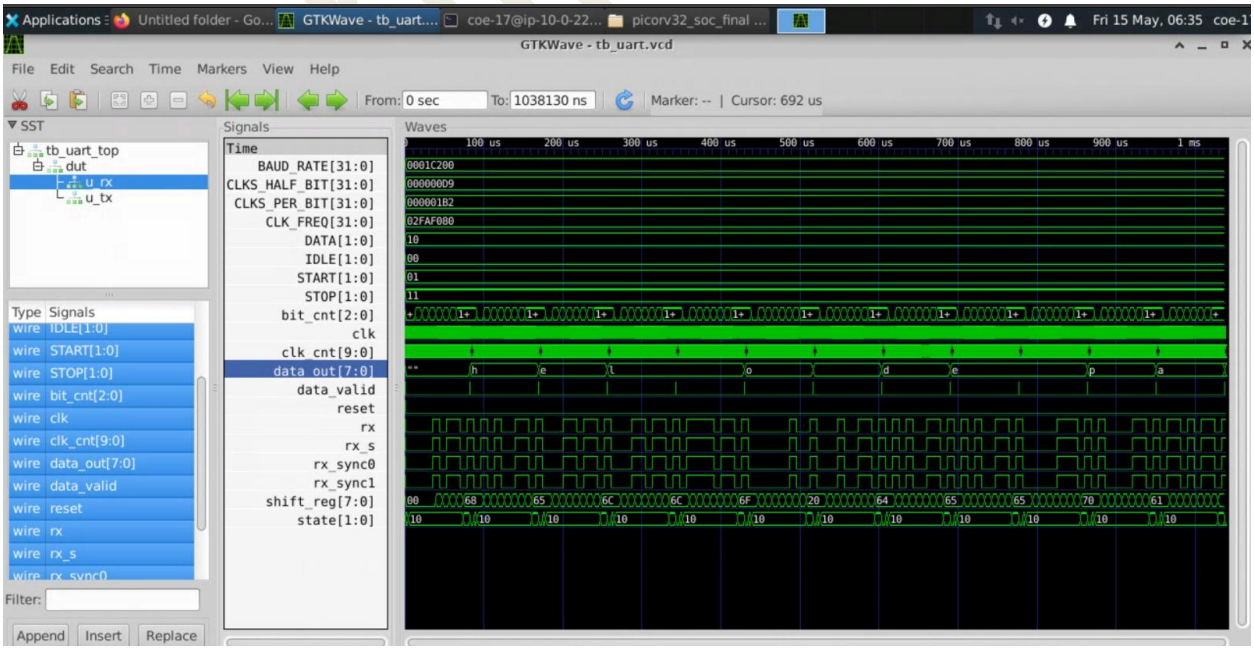
Add these signals:

```
clk
reset
uart_top.tx
uart_top.rx
uart_tx.state
uart_tx.clk_cnt[9:0]
uart_tx.shift_reg[7:0]
uart_tx.bit_cnt[2:0]
uart_rx.state
uart_rx.data_out[7:0]
uart_rx.data_valid
```

Step 3: Measure Bit Period

In GTKWave: 1. Click on the `tx` signal 2. Place a marker at the falling edge of the start bit 3. Place another marker 434 cycles later 4. Verify the bit transitions align with marker spacing (≈ 8.68 µs)

7. Waveform / Verification Insight



TX Side Observations

Signal	What To See
`tx`	Starts HIGH (idle), drops to 0 (start), then 8 data bits LSB-first, then 1 (stop)
`uart_tx.state`	Cycles: IDLE(0) → START(1) → DATA(2) → STOP(3) → IDLE(0)
`uart_tx.clk_cnt`	Counts 0→433 in each bit, then resets
`uart_tx.bit_cnt`	Counts 0→7 during DATA state (8 data bits)
`uart_tx.shift_reg`	Right-shifts each cycle — MSB shifted out, 0 shifted in

RX Side Observations

Signal	What To See
`uart_rx.state`	IDLE → START → DATA → STOP → IDLE
`uart_rx.clk_cnt`	Half-bit wait in START; full-bit wait in DATA/STOP
`uart_rx.shift_reg`	Builds up the received byte bit by bit
`uart_rx.data_valid`	Single-cycle pulse when byte is complete
`uart_rx.data_out`	Should match the transmitted byte

Loopback Timing Delay

In waveforms, notice that `uart_rx.data_valid` pulses approximately:

```
delay = (1 start + 8 data + 1 stop) × 434 cycles = 10 × 434 = 4340 cycles
     ≈ 4340 × 20 ns = 86.8 µs after TX starts
```

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
`data_valid` never pulses	RX not connected to TX in loopback	Check `uart_top.v` for correct TX→RX wire
Received byte is wrong	Bit sampling at edge instead of centre	Verify `CLKS_HALF_BIT` is correctly `CLKS_PER_BIT/2`
Framing error (stop=0)	Clock frequency or baud rate mismatch	Verify parameters: `CLK_FREQ=50_000_000`, `BAUD_RATE=115_200`
First bit always corrupted	2-FF synchronizer not reset properly	Check `rx_sync0/rx_sync1` reset to `1'b1` (idle)

Issue	Likely Cause	Fix
TX sends garbage	`shift_reg` loaded wrong	Verify `valid` pulse triggers `shift_reg <= data` in IDLE state
GTKWave shows flat TX	`valid` never raised in testbench	Check testbench sequence starts the send correctly

Debug Snippet: Manually Decode a UART Byte

For `BAUD_RATE=115200, CLK_FREQ=50MHz`:

```
CLKS_PER_BIT = 434 cycles
1 frame = 10 bits × 434 cycles = 4340 cycles = 86.8 µs
```

In GTKWave: 1. Find falling edge of `tx` (start bit) 2. Jump 217 cycles → confirm `tx=0` (centre of start bit) 3. Jump 434 cycles × 8 → sample each data bit at mid-point 4. Build byte from D0(LSB) to D7(MSB)

9. Summary

Concept	Detail
Protocol	Asynchronous serial, no shared clock
Frame format	8N1: 1 start + 8 data (LSB first) + 1 stop
Baud rate	115200 baud → 434 cycles/bit at 50 MHz
TX FSM	IDLE → START → DATA(×8) → STOP → IDLE
RX FSM	IDLE → START (half-bit wait) → DATA(×8) → STOP → IDLE
Clock safety	2-FF synchronizer prevents metastability on async RX input
Glitch filter	RX rejects false start if RX goes high within half-bit time
Lab result	Byte loopback verified: TX data == RX data_out

UART is the output terminal of this SoC. Every `uart_puts()` call in firmware eventually becomes serial bit transitions on the `uart_tx` pin — visible in simulation as the `[TB] RX byte:` log lines.

Previous: *Sub-module 2 — AXI-Lite*
Next: *Sub-module 4 — Memory-Mapped RISC-V + UART*